

HMS Project Report

1. Student Details

Name: Devansh Malhotra

Roll Number: 23F3003672

Student Mail ID: 23f3003672@ds.study.iitm.ac.in

About Me: I am Devansh Malhotra, from Uttar Pradesh, currently at the Diploma level of the degree. I like to constantly explore the intersections of technology and innovation. It was a great learning experience throughout the course of the project.

2. Project Details

Project Title: **Hospital Management System(HMS)**

Problem Statement:

To design and build a role-based full stack web application that allows hospitals to streamline daily hospital operations by digitally **managing patients, doctors, appointments, and treatment records.**

Approach:

The app uses Flask for backend along with SQLite for database, VueJS for frontend, Redis for caching and Celery for backend jobs like daily reminders or exporting as csv, keeping the structure organized by user roles. It allows patients to easily find and book available slots, doctors to manage their schedules and write prescriptions, and admins to oversee all hospital activities. The app uses caching for better performance. It has some periodic backend jobs to generate and send monthly reports, and send daily reminders to users via email.

3. AI/LLM Declaration

I used **Github Copilot** for assistance in styling(minimal), UI enhancements, writing API documentation, and for email templates and notifications . The extent of AI/LLM usage is around **15–20%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend framework used to build the RESTful APIs and handle server logic
Vue.js	Frontend JavaScript framework used to build the dynamic user interfaces and role based dashboards.
SQLite	Lightweight local database used for securely storing user and hospital data
Bootstrap 5	CSS framework used for responsive design, layout styling, and pre-built UI components
Flask-SQLAlchemy	Object Relational Mapper (ORM) to interact with the SQLite database easily
Flask-Security	User authentication, password hashing, and Role-Based Access Control (Admin, Doctor, Patient)
Celery	Task queue used to run asynchronous background jobs (like CSV exports) and scheduled tasks
Redis	In-memory data store used as a message broker for Celery and for caching API responses

5. Database Schema / ER Diagram

Tables:

- **User** - stores authentication and base profile details (id, email, password, name, active, confirmed_at, fs_uniquifier)
- **Role** - stores role definitions for access control like Admin, Doctor, or Patient (id, name, description)
- **RolesUsers** - association table mapping users to their specific roles (user_id, role_id)
- **Department** - stores hospital departments (id, name, description)
- **Doctor** - stores doctor-specific details linking to a user and department (id, user_id, department_id, qualification, description, experience, is_active)
- **Patient** - stores patient-specific medical and contact details (id, user_id, date_of_birth, phone, address, is_active)
- **DoctorAvailability** - stores time slots when a doctor is available for booking (id, doctor_id, available_date, start_time, end_time)
- **Appointment** - logs bookings made by patients with specific doctors (id, patient_id, doctor_id, appointment_date, appointment_time, status, created_at)
- **Treatment** - records the medical details of a completed appointment (id, appointment_id, visit_type, tests_done, diagnosis, prescription, notes, created_at)
- **ExportJob** - tracks asynchronous CSV export requests (id, user_id, job_type, status, file_path, created_at, completed_at)
- **NotificationLog** - logs automated email alerts and reminders sent to users (id, user_id, appointment_id, notification_type, channel, status, created_at)

Relationships:

- **Many-to-Many** → User ↔ Role (via roles_users table)
- **One-to-One** → User → Doctor
- **One-to-One** → User → Patient

- **One-to-Many** → Department → Doctor
- **One-to-Many** → Doctor → DoctorAvailability
- **One-to-Many** → Doctor → Appointment
- **One-to-Many** → Patient → Appointment
- **One-to-One** → Appointment → Treatment
- **One-to-Many** → User → ExportJob
- **One-to-Many** → User → NotificationLog

Here is the ER Diagram for the App Database: [HMS-ER-DIAGRAM](#)

6. API Resource Endpoints

This app has several endpoints some of which are:

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user credentials and generate role-based session
/api/patient/appointments	POST	Securely book a new appointment and prevent double-booking
/api/doctor/appointments/<int:appointment_id>/treatment	POST	Record patient visit details, diagnosis, tests done, and prescription
/api/patient/doctors/<int:doctor_id>/availability	GET	Fetch real-time, validated available time slots for a specific doctor
/api/patient/export-csv	POST	Trigger a Celery background task to export patient medical history

All endpoints are covered in detail in a YAML API Definition File separately in the zip as `api.yaml`.

7. Video Presentation

Drive Link: <https://drive.google.com/file/d/18MAkloOnvv5frcbuSZdBWrYJovsUcwp2/view>